

# 개발자에서 AI Native로

Claude Desktop으로 완성하는 업무 생산성 혁신

- 서울AI재단 5시간 실습형 워크숍
- 문제 정의부터 PRD, 설계, 구현, 리뷰까지 한 번에 경험한다

---

강사 · 무신사 Agentic AI PM · 서울시립대 / 아주대 AI 부문 겸임교수

# 강사 소개

---

- 무신사 Agentic AI PM
- 서울시립대 / 아주대 AI 부문 겸임교수
- 금융 AutoML Solution / 의료 Computer Vision Ops / 교육

## LLMOps 경험

- 현재 무신사에서 Agent 서비스 기획 진행

### ML

AutoML Solution

금융 도메인

### DL

Computer Vision Ops

의료 도메인

### LLM

LLMOps Service

교육 도메인

### Agent

Agentic AI PM

무신사 (현재)

# 오늘의 큰 질문

---

## AI에게 일을 맡긴다는 것

사람이 무엇을 정의해야 하나

## 문서와 코드의 연결

문서는 어떻게 구현 기준이 되나

## Claude Desktop의 역할

어떤 업무 파트너가 될 수 있나

## 사람의 판단 영역

사람은 어디서 검토해야 하나

# 오늘 만들 결과물

---

## 5시간이 끝나면 작은 제품 개발 패키지가 남는다

- Problem Statement
- PRD v1
- 설계 문서 (Flowchart, Sequence, ERD)
- 민원/FAQ 자동화 프로토타입
- Prompt v2 + 리뷰 노트와 확장 아이디어

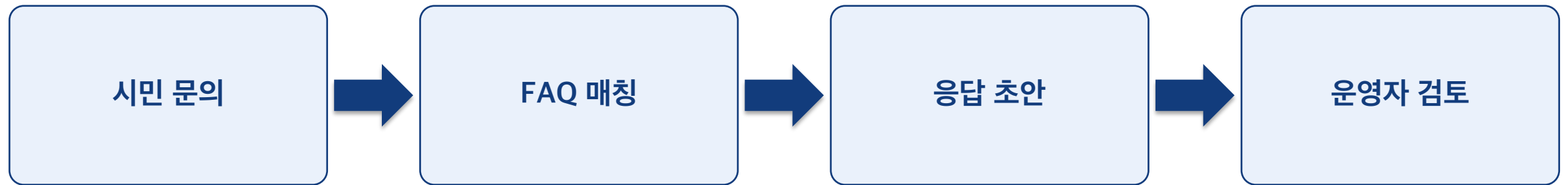
# 5시간 전체 시간표

| 시간          | 세션             | 핵심 내용                                  | 산출물               |
|-------------|----------------|----------------------------------------|-------------------|
| 13:00-13:30 | 환경 세팅 & OT     | Claude Desktop, GitHub clone, 실습 환경 구성 | 실행 환경             |
| 13:30-14:00 | 문제 정의 & 서비스 이해 | FAQ/민원 자동화 시나리오, 사용자 흐름 정의             | Problem Statement |
| 14:00-14:30 | PRD 작성         | 기능 정의, 사용자 플로우, 성공 기준                  | PRD v1            |
| 14:30-14:40 | Break          | 휴식                                     | —                 |
| 14:40-15:10 | 설계 문서화         | 아키텍처, Flow, Sequence, 테이블              | 설계 문서             |
| 15:10-15:50 | 프로토타입 구현 1     | Mock data, API 연결                      | 기능 v1             |
| 15:50-16:00 | Break          | 휴식                                     | —                 |
| 16:00-16:40 | 프로토타입 구현 2     | 응답 생성, UI 연결                           | 기능 v2             |
| 16:40-17:10 | AI 품질 개선       | Prompt 튜닝, Edge case                   | Prompt v2         |
| 17:10-17:30 | 코드 리뷰 & 테스트    | Claude/Codex 리뷰, 리팩토링                  | 리뷰 노트             |
| 17:30-17:50 | Q&A / 확장 논의    | 실무 적용, 확장 방향                           | 인사이트              |
| 17:50-18:00 | Wrap-up        | 핵심 정리, Best Practice                   | 최종 결과             |

슬라이드는 설명 자료이자 다음 작업으로 넘어가는 안내판

# 오늘의 실습 시나리오

---



- 민원/FAQ 자동화 보조 서비스
- 반복 문의를 분류하고 FAQ와 연결
- 응답 초안을 생성한다
- 운영자가 검토하고 확정한다

# 오늘의 핵심 메시지

---

## AI Native는 도구 목록이 아니라 일의 순서다

- 좋은 결과는 좋은 질문보다 좋은 맥락에서 나온다
- 문서가 구현 품질을 결정한다
- 자동화보다 검토 가능한 협업 구조가 먼저다
- 빠른 개발의 핵심은 작게 나누고 자주 검토하기

# 02

## AI Native Workflow와 Claude Desktop

AI를 흐름 안에 배치한다

이 파트의 산출물

- AI Native의 정의와 기존 방식과의 차이
- Claude Desktop에 일을 잘 맡기는 공식
- 프로젝트 폴더 기반 협업 감각



# 기존 업무 방식과의 차이

## Question-based

- 필요한 순간마다 AI에게 질문
- 산출물은 사람이 다시 정리
- 맥락이 매번 끊김

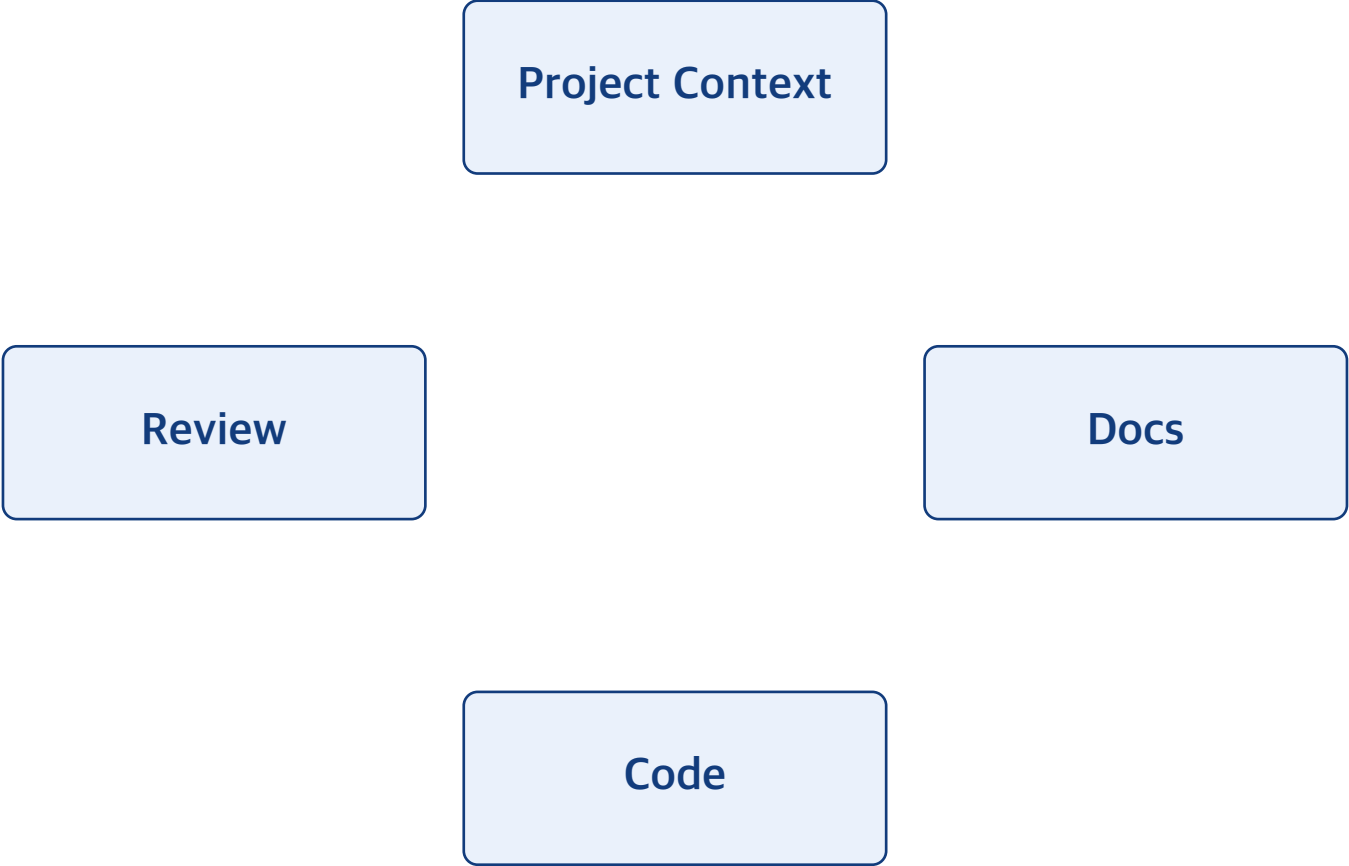
산출물이 다음 산출물의 재료가 되는 구조를 만든다

## Workflow-based

- 산출물이 다음 단계의 입력
- 문서와 코드가 이어짐
- 사람은 기준과 검토 담당

# Claude Desktop의 역할

---



프로젝트 폴더 맥락을 읽고, 문서와 코드를 함께 다루며, 검토 기준을 반영해 개선한다

# Claude에게 일을 잘 맡기는 기본 공식

---



- 역할 — 어떤 관점으로 일할지
- 맥락 — 어떤 문서와 파일을 기준으로 볼지
- 산출물 — 무엇을 만들어야 하는지
- 제약 — 무엇을 하지 말아야 하는지
- 검토 기준 — 성공 여부를 어떻게 판단할지

# 나쁜 요청과 좋은 요청

---

- 나쁜 요청 — '앱 만들어줘'
- 좋은 요청 — 'PRD와 설계 문서를 기준으로 문의 목록 API부터 구현해줘'
- 차이는 구체성보다 작업 단위와 기준

## Before

앱 만들어줘  
쉽게 빠르게  
기준 문서 없음

## After

PRD와 설계 문서를 기준으로  
문의 목록 API부터  
변경 파일과 이유 요약

# 실습 저장소 구조 읽기

---

- docs — 강의 문서와 설계 문서
- claude — Claude Desktop용 맥락과 프롬프트
- apps/api — 백엔드 API
- apps/web — 운영자 UI
- data/mock — FAQ와 문의 Mock 데이터

```
/
├─ docs/
│   ├── 00-course-overview.md
│   ├── 04-prd-faq-automation.md
│   └── 05-architecture-faq-automation.md
├─ claude/
│   ├── CLAUDE.md
│   ├── PROJECT_CONTEXT.md
│   └── prompts/
├─ apps/
│   ├── api/    (Express + TS)
│   └── web/    (React)
└─ data/
    └── mock/
        ├── faqs.json
        └── inquiries.json
```

# 데모 1 · 프로젝트 요약과 4인 역할 분담

## AI PM

문제 정의, PRD

비목표/리스크 관리

## UX Builder

사용자 흐름, UI

Flowchart 담당

## Backend Engineer

API, 데이터 모델

Sequence/ERD 담당

## Quality Engineer

테스트, Edge case

리뷰 노트 담당

Claude Desktop에 프로젝트 폴더를 열고 `claude/PROJECT_CONTEXT.md`를 먼저 읽힌다

참고 — `docs/13-ai-developer-intern-lab-plan.md`

개발자에서 AI Native로 · 서울AI재단 5시간 워크숍

# 03

## 문제 정의 & 서비스 이해

기능 목록 이전에 문제와 사용자

이 파트의 산출물

- Problem Statement
- 사용자별 불편 분리
- 자동화 가능/제외 범위

# 자동화할 수 있는 일과 하면 안 되는 일

| 할 수 있는 일                                                                                                  |
|-----------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• 분류</li><li>• FAQ 매칭</li><li>• 응답 초안</li><li>• 검토 포인트 제안</li></ul> |

| 조심할 일                                                                                           |
|-------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• 정책 판단</li><li>• 민감 민원 처리</li><li>• 개인정보 포함 문의</li></ul> |

| 하지 말아야 할 일                                                        |
|-------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• 운영자 승인 없는 자동 발송</li></ul> |

자동화 가능성만 보지 말고, 자동화 위험 영역도 함께 정한다



# 사용자 관점으로 문제 보기

---

## 시민

빠르고 정확한 답변

## 운영자

반복 답변 시간 단축

검토 가능한 초안

## 관리자

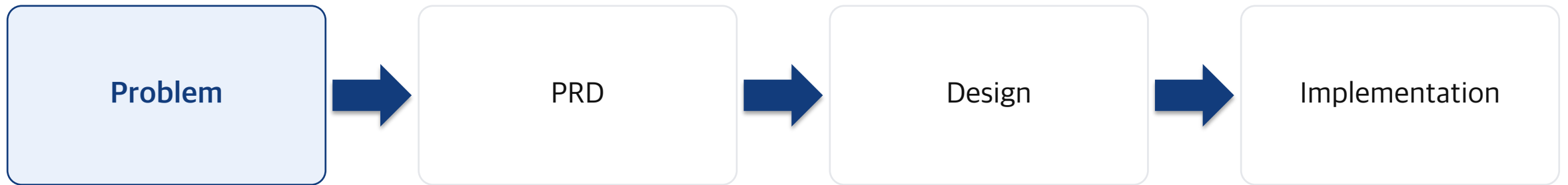
처리량과 품질 편차

리스크 모니터링

문제 정의는 기능 목록이 아니라, 누가 어떤 불편을 겪는지 분리해서 보는 일

# Problem Statement의 역할

---



- 무엇이 문제인지 한 문단으로 정리한다
- 왜 지금 해결해야 하는지 설명한다
- 해결 범위와 비범위를 구분한다
- 이후 PRD와 설계의 기준이 된다

# 문제 정의 프롬프트 예시

## Prompt

- 역할 — AI PM
- 맥락 — 민원/FAQ 자동화
- 산출물 — Problem Statement
- 제약 — 완전 자동 발송은 제외
- 기준 — 공공기관 실무에 맞는 표현

## claude/prompts/01-problem-framing.md

너는 공공기관 민원 운영을 이해하는 AI PM이야.

민원/FAQ 자동화 보조 서비스를 위한  
Problem Statement 초안을 만들어줘.

포함:

- 시민/운영자/관리자별 불편
- 자동화 가능 범위
- 자동화 제외 범위
- 성공의 정의

제약: 완전 자동 발송은 제외한다.

# 실습 1 · Problem Statement 만들기

## AI PM이 초안 → 나머지 3명이 누락 관점 리뷰

- ☐ claude/prompts/01-problem-framing.md를 사용한다
- ☐ 운영자, 시민, 관리자 문제를 분리한다
- ☐ 자동화 가능 범위와 제외 범위를 함께 적는다
- ☐ 결과를 docs 문서와 비교한다

# 문제 정의 결과 리뷰

지금은 멋진지보다 다음 단계로 넘길 수 있는지를 본다

- ☐ 문제가 기능으로만 쓰이지 않았는가
- ☐ 사용자별 불편이 분리되었는가
- ☐ 자동화 제외 범위가 있는가
- ☐ 이후 PRD로 이어질 만큼 명확한가

# 04

## PRD 작성

구현 기준이자 의사결정 기준

이 파트의 산출물

- MVP PRD v1
- 비목표와 성공 지표
- Leadership / Designer / Engineer 리뷰

# 좋은 PRD의 최소 구성

---

## 문제 정의

왜 만드는가

## 사용자

누가 쓰는가

## 핵심 기능

MVP 범위

## 비목표

하지 않는 것

## 성공 지표

어떻게 판단하나

## 리스크

오픈 질문

오늘은 1일 내 프로토타입 가능한 최소 PRD를 만든다

# MVP 범위 제한이 중요한 이유

---

AI는 범위를 넓히는 데 능하다 — 좁히는 것이 PM의 역할

- 실습 시간은 제한되어 있다
- MVP는 작아야 완성된다
- 완성된 작은 기능이 확장 논의의 출발점이다



# 민원/FAQ 자동화 MVP

## 운영자가 문의를 보고 AI 초안을 검토하는 흐름까지

- ☐ Mock 데이터 기반 문의 목록
- ☐ 문의 상세 보기
- ☐ FAQ 매칭
- ☐ 응답 초안 생성
- ☐ 운영자 검토 포인트 표시

# 성공 기준은 어떻게 잡을까

| Metric                                                                                 | Quality                                                                       | Safety                                                                          |
|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• 응답 초안 작성 시간 단축</li><li>• 반복 문의 처리 속도</li></ul> | <ul style="list-style-type: none"><li>• 운영자 검토 가능성</li><li>• 톤과 정확성</li></ul> | <ul style="list-style-type: none"><li>• FAQ 미매칭 처리</li><li>• 확정 표현 제한</li></ul> |

AI 기능의 성공 기준은 정확도 하나로 끝나지 않는다

# PRD 작성 프롬프트 예시

## Prompt

- 참고 문서 — Problem Statement
- 목표 — MVP PRD
- 포함 항목 — 사용자, 기능, 비목표, 성공 지표
- 제약 — 1일 내 프로토타입 가능한 수준

## claude/prompts/02-prd-writing.md

Problem Statement를 입력으로 사용해  
민원/FAQ 자동화의 MVP PRD를 작성해줘.

포함:

- 사용자 (시민/운영자/관리자)
- 핵심 기능 (분류/매칭/초안/검토)
- 비목표 (자동 발송, 관리자 콘솔)
- 성공 지표 (Metric/Quality/Safety)
- 리스크 / 오픈 질문

범위: 1일 내 프로토타입 가능한 수준

# 실습 2 · PRD v1 만들기

## Problem Statement → PRD → 3관점 리뷰 준비

- ☐ Problem Statement를 입력으로 사용한다
- ☐ docs/templates/prd-template.md 구조를 따른다
- ☐ MVP 범위를 작게 유지한다
- ☐ 비목표와 리스크를 반드시 포함한다
- ☐ Leadership / Designer / Engineer 관점으로 1차 리뷰

# PRD는 세 관점으로 리뷰한다

---

## Leadership

회사에 왜 필요한가

Business Value

## UI/UX Designer

사용자가 잘 쓸 수 있는가

User Experience

## Engineer

구현 가능한가

Technical Feasibility

# 05

## 설계 문서화

PRD를 구현 가능한 구조로 옮긴다

이 파트의 산출물

- Flowchart / Sequence / ERD
- API 계약과 데이터 모델
- AI Layer 분리 구조

# Mermaid로 먼저 그릴 3가지

---

## Flowchart

사용자 흐름

어떤 순서로 쓰는가

## Sequence Diagram

호출 순서

Web/API/AI Layer

## ERD

데이터 구조

Inquiry/FAQ/Recommendation

## Flowchart · 사용자 흐름 먼저 보기

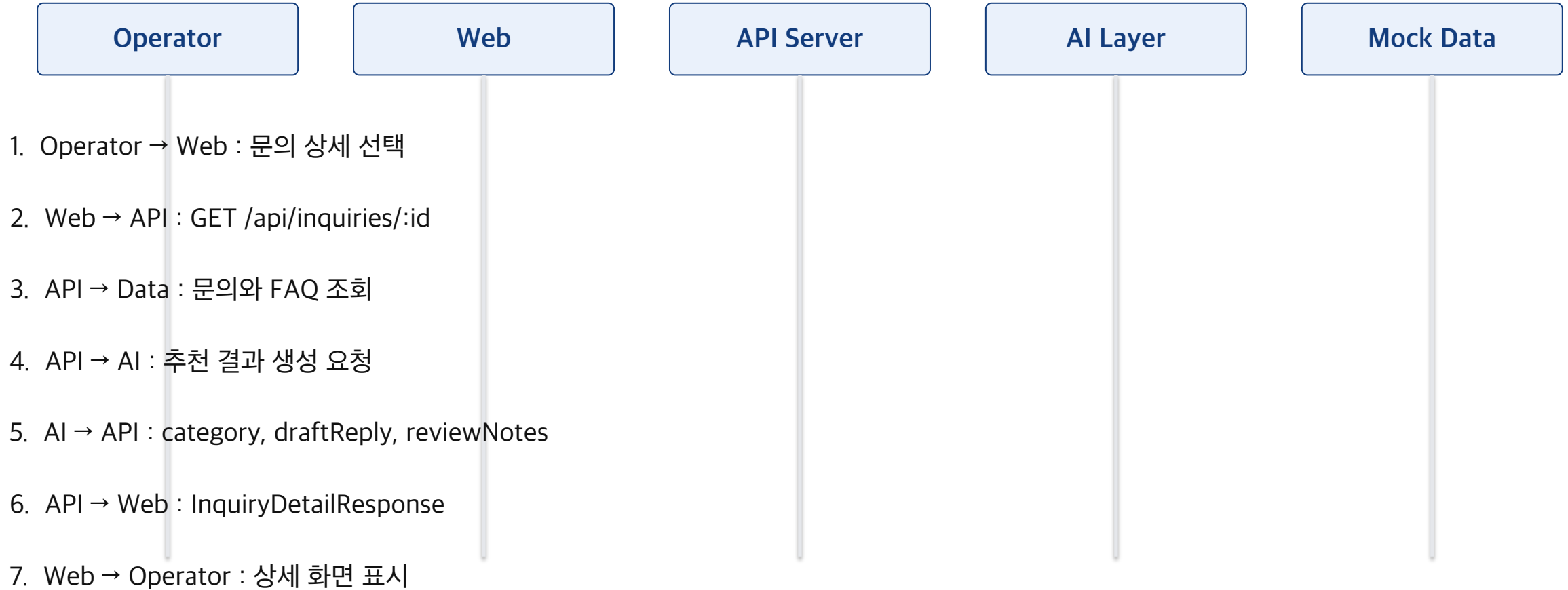
---



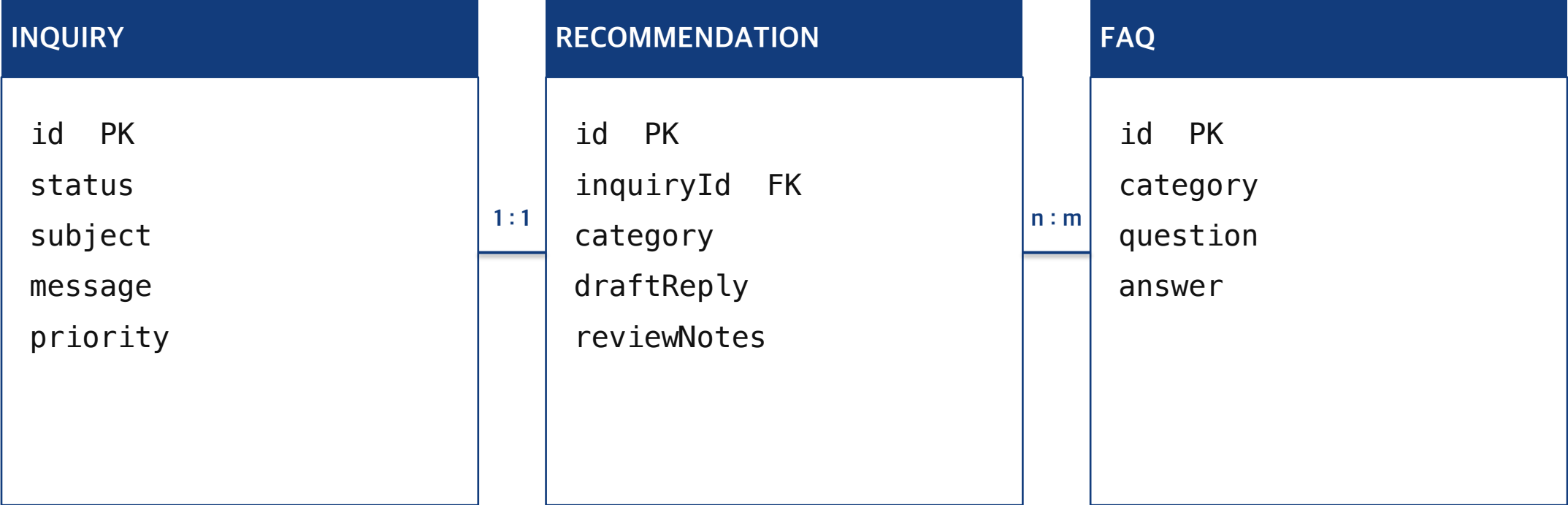
- 자동 발송이 아니라 사람 검토를 반드시 거친다



# Sequence Diagram · API 호출 순서 보기



# ERD · 데이터 구조 보기



# API 설계 초안

| Method | Path                              | 역할                 |
|--------|-----------------------------------|--------------------|
| GET    | /health                           | 서버 상태 확인           |
| GET    | /api/faqs                         | FAQ 전체 조회          |
| GET    | /api/inquiries                    | 문의 목록              |
| GET    | /api/inquiries/:id                | 문의 상세 + 추천 결과      |
| POST   | /api/inquiries/:id/recommendation | 분류·매칭·초안·검토 포인트 생성 |

오늘 데모는 목록 / 상세 / 추천 생성 정도면 충분

# AI Layer를 분리하는 이유

---

- Mock 규칙 기반으로도 데모 가능
- 나중에 LLM API로 교체 가능
- 분류, 매칭, 응답 생성을 따로 개선 가능
- 품질 평가 지점을 분리할 수 있음

## Now

Rule-based AI Layer

data/mock 활용

강의 중 안정 동작

## Later

Claude API / 다른 LLM 교체

RAG / 정책 검색 추가

평가 데이터 축적

# 설계 문서 작성 프롬프트

## Prompt

- 역할 — Tech Lead
- 기준 문서 — PRD
- 산출물 — architecture, API, data model, AI flow
- 제약 — 강의용으로 이해하기 쉬운 구조

## claude/prompts/03-system-design.md

PRD를 입력으로 사용해 설계 문서를 작성해줘.

포함:

- Mermaid Flowchart (사용자 흐름)
- Mermaid Sequence (Web/API/AI 호출 순서)
- Mermaid ERD (Inquiry/FAQ/Recommendation)
- API Contract (Method, Path, Body)
- AI Layer 입출력 명세

구조: 강의용으로 이해하기 쉽게.

# 실습 3 · 설계 문서 만들기

PRD → Mermaid → 설계 → 구현 순서

- ☐ PRD를 입력으로 사용한다
- ☐ Flowchart, Sequence, ERD를 작성한다
- ☐ API, 데이터 모델, AI Layer를 정리한다
- ☐ 구현 순서를 함께 뽑는다
- ☐ UX Builder는 Flow, Backend는 Sequence/ERD, Quality는 상태/테스트 흐름

# 구현 전 최종 확인

---

이 네 가지가 분명해야 Claude가 어긋나지 않는다

- ☐ 무엇을 먼저 만들 것인가
- ☐ 어떤 Mock 데이터를 쓸 것인가
- ☐ API 응답 형태는 무엇인가
- ☐ UI는 어떤 정보를 보여줄 것인가

# 06

## 프로토타입 구현 1 API와 Mock Data

작은 단위로 자주 실행 확인

이 파트의 산출물

- MVP API 5개
- Mock 데이터 연결
- 기능 v1 (서버 실행 가능)



# Mock Data의 역할

---

- 네트워크나 DB 없이 데모 가능
- 입력과 출력 예시를 고정
- AI 결과 품질 비교가 쉬움
- 강의 중 실패 리스크 감소

```
// data/mock/faqs.json
[
  {
    "id": "faq-001",
    "category": "민원처리",
    "question": "민원 처리 기간은 얼마인가요?",
    "answer": "일반 민원은 7일 이내..."
  }
]

// data/mock/inquiries.json
[
  {
    "id": "inq-001",
    "subject": "민원 처리 기간 문의",
    "status": "open"
  }
]
```

# API 구현 요청 예시

## Prompt

- 기준 문서 — docs/04-prd-faq-automation.md
- 기준 문서 — docs/05-architecture-faq-automation.md
- 범위 — apps/api MVP API
- 데이터 — data/mock 사용
- 산출물 — 변경 파일과 이유 요약

## claude/prompts/04-implementation.md

docs/04-prd-faq-automation.md 와  
docs/05-architecture-faq-automation.md 를 기준으로  
apps/api 의 MVP API를 구현해줘.

Mock 데이터는 data/mock/ 을 사용한다.

구현 후 변경된 파일 목록과  
각 파일의 변경 이유를 요약해줘.

# 문의 목록 API

---

- GET /api/inquiries
- id, status, subject, channel, priority
- 운영자 화면의 첫 데이터 소스

## GET /api/inquiries

```
[
  {
    "id": "inq-001",
    "status": "open",
    "subject": "민원 처리 기간 문의",
    "channel": "web",
    "priority": "normal"
  },
  {
    "id": "inq-002",
    "status": "in_review",
    "subject": "개인정보 정정 요청",
    "channel": "phone",
    "priority": "high"
  }
]
```

# 문의 상세 API

---

- GET /api/inquiries/:id
- 원문 + 추천 결과를 함께 반환
- UI 상세 패널의 데이터 소스

**GET /api/inquiries/inq-001**

```
{
  "inquiry": {
    "id": "inq-001",
    "subject": "민원 처리 기간 문의",
    "message": "제가 5월 1일에 신청한..."
  },
  "recommendation": {
    "category": "민원처리",
    "matchedFaqs": ["faq-001"],
    "draftReply": "안녕하세요, 시민님...",
    "reviewNotes": [
      "실제 접수일 확인이 필요"
    ]
  }
}
```

# 추천 생성 API

---



- POST /api/inquiries/:id/recommendation
- 카테고리 분류 / FAQ 매칭 / 응답 초안 / 검토 포인트
- 오늘 데모의 핵심 기능

## 데모 2 · API 서버 실행

---

- 의존성 설치
- API 개발 서버 실행
- /health 확인
- 문의 목록 API 확인

```
$ npm install
```

```
$ npm run dev:api
```

```
▶ api server listening on :4000
```

```
$ curl http://localhost:4000/health  
{ "status": "ok" }
```

```
$ curl http://localhost:4000/api/inquiries  
[ ... ]
```

# 실습 4 · API 응답 확인하기

## 추천 결과의 reviewNotes가 함께 오는지 본다

- ☐ /health
- ☐ /api/faqs
- ☐ /api/inquiries
- ☐ /api/inquiries/:id
- ☐ 추천 결과의 reviewNotes 확인

# API 구현 리뷰 포인트

---

## 데이터 / 라우트 / AI 로직이 분리되어 있는가

- ☐ Mock 데이터가 분리되어 있는가
- ☐ AI 로직이 별도 함수로 분리되어 있는가
- ☐ 없는 문의 id에 404를 반환하는가
- ☐ 응답 형태가 UI에서 쓰기 쉬운가



# 07

## 프로토타입 구현 2 UI와 응답 생성

운영자 콘솔로서의 화면

이 파트의 산출물

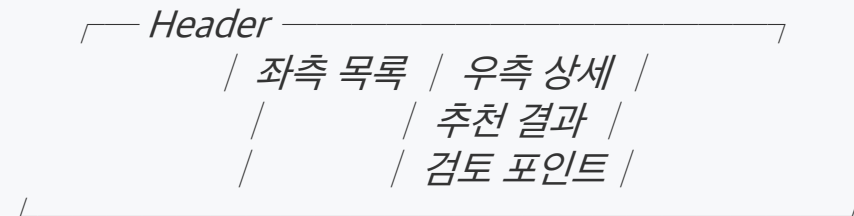
- 목록 / 상세 2단 레이아웃
- 추천 결과 + 검토 포인트
- 기능 v2 (E2E 데모 완성)

# UI 구성 요소

---

- 상단 — 서비스 제목과 목적
- 좌측 — 문의 목록
- 우측 — 상세 문의
- 우측 내부 — 응답 초안, 검토 포인트

*wireframe placeholder*



# UI 구현 요청 예시

## Prompt

- 범위 — apps/web 운영자 화면
- API 계약 유지
- 목록/상세 2단 레이아웃
- 강의용 가독성 우선

## claude/prompts/04-implementation.md (UI)

apps/web 에서 운영자 화면을 만들어줘.

API 계약은 apps/api 와 정확히 맞추고,  
목록 / 상세 2단 레이아웃을 사용한다.

상세 패널에는

- 문의 원문
  - 카테고리 / FAQ 매칭
  - 응답 초안
  - 검토 포인트
- 를 함께 표시한다.

코드는 강의에서 읽기 쉽게 작성해줘.

## 데모 3 · Web 앱 실행

---

- Web 개발 서버 실행
- 문의 목록 → 선택 → 상세
- 응답 초안과 검토 포인트 확인

```
$ npm run dev:web
```

```
▶ web running on http://localhost:5173
```

- 브라우저 열기
- 좌측 목록에서 문의 선택
- 우측 상세 패널 확인
- 추천 결과의 reviewNotes 확인

## 실습 5 · UI에서 문의 흐름 확인

### 이때부터는 품질 판단의 눈을 함께 가져간다

- ☐ 각 문의를 선택한다
- ☐ 카테고리가 어떻게 바뀌는지 본다
- ☐ FAQ 매칭 수를 확인한다
- ☐ 응답 초안의 톤을 확인한다
- ☐ 검토 포인트가 적절한지 본다

# 응답 초안에서 봐야 할 것

## 정책상 확정할 수 없는 것을 확정하지 않는가

- ☐ 친절한가
- ☐ 과도하게 확정하지 않는가
- ☐ FAQ 근거와 맞는가
- ☐ 추가 확인이 필요한 부분을 남기는가

# 기능 v2의 기준

v1

- API 응답만 확인
- 추천 결과를 raw로 봄

v2

- 목록 → 상세 흐름
- 추천 결과 / 초안 / 검토 포인트
- 데모 중 끊기지 않음

거창한 완성본이 아니라, 핵심 흐름이 끊기지 않으면 충분

# 08

## AI 품질 개선

Generate → Review → Tune → Test

이 파트의 산출물

- 분류 / 응답 / 검토 품질 진단
- Edge case 대응 룰
- Prompt v2



# 품질 개선의 세 축

---

분류 품질

+

응답 품질

+

검토 품질

- 문의를 잘 분류하는지
- 답변 초안이 적절한지
- 운영자 확인 포인트를 잘 남기는지

# Edge Case 예시

---

## FAQ에 없는 문의

근거 없는 답변 위험

## 감정이 강한 민원

톤 조절 필요

## 정책 판단 필요

담당자 확인 유도

## 개인정보 포함

로그/응답 처리 주의

## 여러 주제 혼재

분류 모호성

—

일부러 넣어보는 습관

# 프롬프트 튜닝 방향

## Before

- 단정적인 답변
- FAQ 없어도 추측
- 검토 포인트 누락

## After

- 모르는 것은 모른다
- 정책 확인 필요 문구
- 검토용 초안임을 명시
- 민감 문의는 담당자 확인 유도

답할 수 있는 것과 확인해야 하는 것을 구분하게 만든다

# Prompt v2 요청 예시

## Prompt v2

- 분류 정확도 향상
- FAQ 미매칭 처리
- 과도한 확정 표현 줄이기
- 운영자 검토 포인트 강화
- 공공기관에 맞는 톤 유지

## claude/prompts/05-prompt-tuning.md

현재 분류 결과와 응답 초안에서  
다음 문제가 보인다:

- FAQ 미매칭 시 무리한 추측
- 정책 판단 영역의 단정
- 검토 포인트 누락

Prompt v2에서는

- 모르는 것은 모른다고 남기게
- 정책 확인 필요 문구를 포함하게
- 검토용 초안임을 명시하게
- 공공기관 어조를 유지하게

수정해줘.

# 실습 6 · 응답 품질 개선하기

---

'더 좋게'가 아니라 '어떤 기준으로 더 좋게'

- ☐ 문의 하나를 선택한다
- ☐ 응답 초안의 문제를 찾는다
- ☐ Claude에게 개선 기준을 준다
- ☐ 변경 후 결과를 비교한다

# 품질 개선 결과 공유

---

## 좋아진 점만 보지 말고 새 부작용도 본다

- ☐ 어떤 문제가 있었는가
- ☐ 어떤 지시를 추가했는가
- ☐ 결과가 어떻게 달라졌는가
- ☐ 새로 생긴 부작용은 없는가

# 09

## 코드 리뷰 & 테스트

동작한다고 충분하지 않다

이 파트의 산출물

- PRD 기준 리뷰 노트
- 최소 테스트 케이스
- Fix Now / Later 분리

# Claude / Codex 리뷰 요청 방법

## Review Prompt

- 버그와 회귀 중심으로 본다
- PRD와 설계 문서를 기준으로 비교
- 테스트가 필요한 지점을 찾는다
- 개선 제안과 실제 수정 범위를 분리

## Review Request Template

현재 apps/api, apps/web 코드를  
다음 기준으로 리뷰해줘:

- docs/04-prd-faq-automation.md 와의 일치 여부
- docs/05-architecture-faq-automation.md 와의 일치 여부
- 발견한 버그와 회귀 가능성
- 누락된 테스트 케이스

출력은

- 1) 발견 사항 / 위험도
- 2) 지금 고쳐야 할 것
- 3) 나중에 다룰 것

으로 분리해줘.



# 최소 테스트 케이스

---

## 케이스

문의 목록 조회

존재하지 않는 문의 id

FAQ 미매칭 문의

응답 초안 생성

## 기대 결과

비어 있지 않다

404 응답

무리한 답변을 만들지 않는다

검토 포인트가 함께 포함된다

AI 기능에서는 실패했을 때의 행동이 더 중요

## 실습 7 · 리뷰 노트 만들기

---

### 리뷰의 목적은 비판이 아니라 다음 액션

- ☐ Claude 또는 Codex에 리뷰를 요청한다
- ☐ 발견된 리스크를 정리한다
- ☐ 바로 고칠 것과 나중에 할 것을 나눈다
- ☐ 리뷰 노트를 산출물로 남긴다

# 코드 리뷰에서 남겨야 할 질문

---

## PRD 부합도

기능이 PRD 목표와 맞는가

## 사용자 판단

사용자가 실제로 판단하기 쉬운가

## 위험한 자동화

숨어 있지 않은가

## 구조적 견고성

데이터/모델 변경에 버틸 수 있는가

# 10

## 실무 확장 & Wrap-up

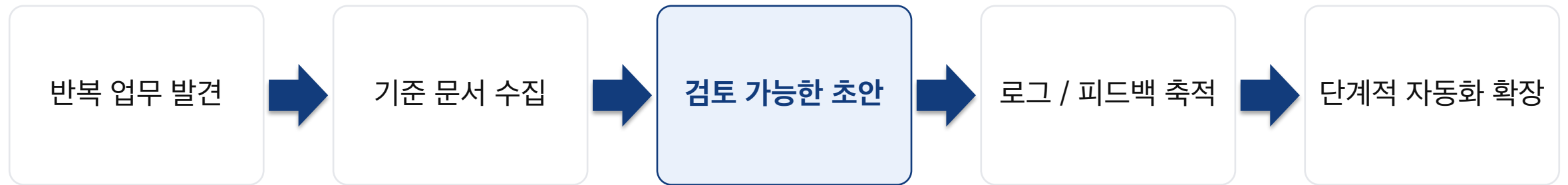
Prototype → Production

이 파트의 산출물

- 실무 도입 로드맵
- AI PM / 개발자 Best Practice
- 오늘의 한 줄 요약

# 조직에 AI 기능을 도입할 때의 순서

---



- 처음부터 완전 자동화를 목표로 잡지 않는다
- 사람이 검토 가능한 초안 생성부터 시작

# AI PM 관점의 Best Practice

---

- 문제를 작게 자른다
- 문서를 먼저 만든다
- AI에게 기준을 준다
- 결과를 사람 기준으로 검토한다
- 반복 개선을 운영 구조로 만든다

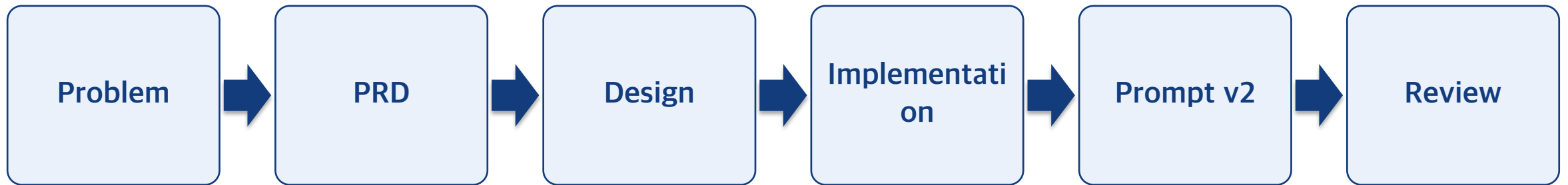
# 개발자 관점의 Best Practice

---

- 작은 단위로 구현 요청
- 실행 가능한 상태 자주 확인
- 타입과 API 계약 유지
- Mock 데이터로 먼저 검증
- 리뷰와 테스트를 마지막에 몰지 않기

# 오늘 만든 흐름을 다시 정리하면

---



- 차이는 이 과정을 Claude Desktop과 함께
- 문서와 코드가 이어지는 방식으로 진행했다는 점



# Q&A · 확장 논의

---

## 실제 LLM API

붙이면 무엇이 달라질까

## RAG

붙이면 어떤 구조가 필요할까

## 운영자 승인 플로우

어떻게 설계할까

## 개인정보와 로그

어떻게 관리할까

# 수강생이 가져갈 수 있는 프롬프트 자산

---

- 문제 정의 / PRD / 설계 / 구현 / 품질 / 리뷰
- 각자 업무 도메인에 맞게 변형해서 사용

```
claude/  
├─ CLAUDE.md  
├─ PROJECT_CONTEXT.md  
└─ prompts/  
    ├─ 01-problem-framing.md  
    ├─ 02-prd-writing.md  
    ├─ 03-system-design.md  
    ├─ 04-implementation.md  
    ├─ 05-prompt-tuning.md  
    └─ 06-review-checklist.md
```

## 오늘의 한 줄 요약

---

AI Native는 AI에게 정답을 묻는 방식이 아니라,  
AI와 함께 일을 끝내는 구조를 만드는 방식이다.

# 마지막 체크리스트

## 도구 사용법보다 흐름을 자기 업무에 적용하는 능력

- ☐ Claude에게 맥락을 먼저 읽혔는가
- ☐ 문서가 구현 기준으로 쓰였는가
- ☐ 자동화 제외 범위를 정했는가
- ☐ 운영자 검토 포인트가 남았는가
- ☐ 다음 확장 방향을 설명할 수 있는가

# 마무리

---

- 질문 — 자유롭게
- 실무 적용 아이디어 공유